

Students:

Syed Sualeh Abbas 10303

Burair Haidar Inayatullah

TASK 1:

CODE:

```
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1); // Red
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2); // Green
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3); // Blue

// Initialize all channels to 999 (LEDs off for common anode)
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 999);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 999);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, 999);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // ===== RED LED =====
    // Fade in RED (decrease PWM value = brighter)
    for(uint16_t brightness = 999; brightness > 0; brightness--)
    {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, brightness);
        HAL_Delay(1);
    }

    // Fade out RED (increase PWM value = dimmer)
    for(uint16_t brightness = 0; brightness <= 999; brightness++)
    {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, brightness);
        HAL_Delay(1);
    }
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 999); // Ensure RED is fully off

    // ===== GREEN LED =====
    // Fade in GREEN
    for(uint16_t brightness = 999; brightness > 0; brightness--)
    {
        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, brightness);
        HAL_Delay(1);
    }
```

```

// Fade out GREEN
for(uint16_t brightness = 0; brightness <= 999; brightness++)
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, brightness);
    HAL_Delay(1);
}
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 999); // Ensure GREEN is fully off

// ===== BLUE LED =====
// Fade in BLUE
for(uint16_t brightness = 999; brightness > 0; brightness--)
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, brightness);
    HAL_Delay(1);
}

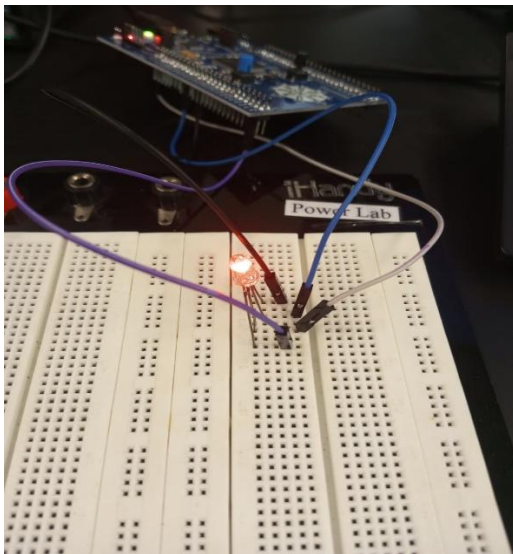
// Fade out BLUE
for(uint16_t brightness = 0; brightness <= 999; brightness++)
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, brightness);
    HAL_Delay(1);
}
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, 999); // Ensure BLUE is fully off

}
/* USER CODE END 3 */
/* USER CODE END 3 */

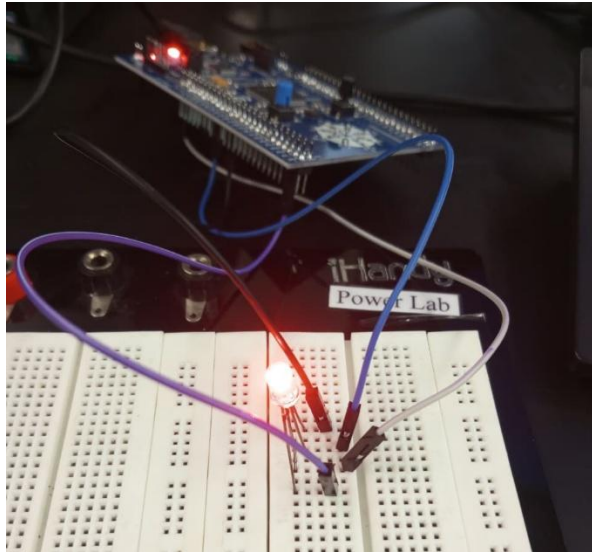
```

OUTPUT PICTURES:

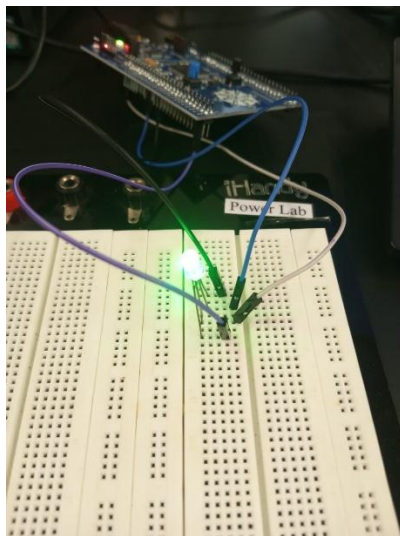
FADING RED:



BRIGHT RED:



OTHER COLORS:



TASK 2:

CODE:

```
// Start PWM for both motors
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1); // Right Motor PWM
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2); // Left Motor PWM
// Initialize motors stopped
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    // ===== Move Forward =====
    // Right Motor: DIR1=HIGH, DIR2=LOW, PWM=150
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET); // D8 (DIR1) = HIGH
```

```

HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_RESET); // D12 (DIR2) = LOW
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 150); // Speed

// Left Motor: DIR1=HIGH, DIR2=LOW, PWM=150
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET); // D7 (DIR1) = HIGH
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_RESET); // D6 (DIR2) = LOW
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 150); // Speed
HAL_Delay(3000); // Forward for 3 second
// ===== Stop =====
// Right Motor: DIR1=HIGH, DIR2=HIGH
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET); // D8 = HIGH
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET); // D12 = HIGH
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
// Left Motor: DIR1=HIGH, DIR2=HIGH
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET); // D7 = HIGH
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET); // D6 = HIGH
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
HAL_Delay(1000); // Stop for 1 second
// ===== Move Backward =====
// Right Motor: DIR1=LOW, DIR2=HIGH, PWM=150
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_RESET); // D8 = LOW
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET); // D12 = HIGH
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 150);
// Left Motor: DIR1=LOW, DIR2=HIGH, PWM=150
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_RESET); // D7 = LOW
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET); // D6 = HIGH
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 150);
HAL_Delay(3000); // Backward for 3 second
// ===== Stop =====
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);

HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET);
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);

HAL_Delay(2000);
}

```

RESULT WAS SHOWN IN LAB

EVALUATION QUESTIONS:

(a) In this lab, the duty cycle of the PWM signal determines how much average voltage is applied to the DC motor, which directly controls its speed. When we increase the duty cycle (by increasing the CCR value), the signal stays HIGH for a longer time in each period, so more power is delivered to the motor and it spins faster. When we decrease the duty cycle, the HIGH time becomes shorter, the average voltage decreases, and the motor slows down. At 0% duty cycle the motor stops because it receives no effective voltage, and at a high duty cycle (close to 100%) it runs at maximum speed. The frequency remains the same; only the pulse width changes.

(b) To generate a 2 kHz PWM signal with a 72 MHz timer clock, we use

$$f_{PWM} = \frac{f_{timer}}{(PSC+1)(ARR+1)}.$$

First, the total counts needed are $72\,000\,000/2\,000 = 36\,000$. We choose a convenient prescaler so the timer runs at an easy frequency. If we set $PSC = 71$, the timer clock becomes 1 MHz. The counter must then count 500 ticks for one PWM period, so $ARR = 499$. This produces a 2 kHz PWM signal, and the duty cycle for motor speed control can be adjusted by changing the CCR value between 0 and 499.